

POTW Deployment Guide

A click-by-click guide for Namecheap, Resend, Render, and PostgreSQL

Prepared for the Purdue Math Club Problem of the Week app

Security rule: Never paste a database password, Resend API key, JWT secret, or complete connection URL into GitHub, screenshots, Discord, or chat. Put secrets only in Render environment variables.

Contents

1	What you are deploying	2
2	Before clicking anything	2
3	Save and push the project	2
4	Namecheap and Resend email setup	2
5	Create Render PostgreSQL	3
6	Unverified signup behavior	4
7	Run the database migrations	4
8	Create the Render backend	5
9	Create the Render frontend	6
10	Connect the two services	6
11	First production test	6
12	Common problems	7
13	After launch	7
14	Useful official documentation	7

1 What you are deploying

The repository is a single project (a monorepo), but Render will run two services from it:

- **Static Site:** builds the React files in `src/` and serves the website.
- **Web Service:** runs the Express API in `server/`. It handles login, email, submissions, grading, hints, and database access.
- **PostgreSQL:** stores users, problems, submissions, grades, seasons, and hint requests.

The request path is: browser → React Static Site → Express Web Service → PostgreSQL.

2 Before clicking anything

Have these ready:

1. A GitHub repository containing `src/`, `server/`, `db/`, `public/`, and `package.json`.
2. A Namecheap account and the domain `purdue-math-potw.com`.
3. A Resend account.
4. A Render account connected to the GitHub account.
5. PostgreSQL command-line tools (`psql`) installed locally for one-time migrations.

The repository must not contain `.env`, `node_modules/`, or `dist/`.

3 Save and push the project

Open PowerShell and run:

```
cd "C:\textbackslash Users\textbackslash YOUR\_NAME\textbackslash Documents\textbackslash GitHub\textbackslash textback
git add .
git commit -m "Prepare POTW for deployment"
git push origin main
```

If Git asks for a login, complete the GitHub sign-in. If the push fails, stop and fix that before continuing.

4 Namecheap and Resend email setup

Step Create the Resend domain entry

1. Go to <https://resend.com> and sign in.
2. In the left menu click **Domains**.
3. Click **Add Domain**.
4. Enter `purdue-math-potw.com` and click **Add**.
5. Leave this Resend page open. It displays the DNS records you must copy.

Step Add DNS records in Namecheap

1. Go to <https://www.namecheap.com> and sign in.
2. Click **Domain List** in the left sidebar.
3. Find `purdue-math-potw.com` and click **Manage**.
4. Click the **Advanced DNS** tab.
5. Under **Host Records**, click **Add New Record**.
6. For every record shown by Resend, choose the same type and paste its exact Host and Value.
7. Set TTL to **Automatic**, then click the green checkmark/save icon.

For a Namecheap Host field, use `@` when Resend means the root domain. If Resend gives a full hostname, Namecheap may only need the part before `.purdue-math-potw.com`. Do not guess: use Resend's displayed values. Do not create a second SPF record if one already exists.

Step Verify in Resend

1. Return to Resend **Domains**.
2. Click `purdue-math-potw.com`.
3. Click **Verify DNS Records**.
4. Wait until the domain status says **Verified**.

Step Create the Resend API key

1. In Resend click **API Keys**.
2. Click **Create API Key**.
3. Name it POTW Production.
4. Select **Sending access** and restrict it to `purdue-math-potw.com` when offered.
5. Click **Create Key**.
6. Copy the complete key beginning with `re_`. Resend only displays it once.

5 Create Render PostgreSQL

1. Go to <https://dashboard.render.com>.
2. Click **New** in the upper-right corner.
3. Select **PostgreSQL**.
4. Name it `potw-db`.
5. Choose the same region you will use for the backend (Ohio is appropriate for Purdue users).
6. Choose a paid production plan if you need reliable always-on production behavior.

9. Open the database, click **Connect** , and copy the **External Database URL** for local migrations.

6 Unverified signup behavior

To test this after deployment:

1. Register a username and email, but do not click its verification link.
2. Register again with the same username and email.
3. Confirm the second registration succeeds and a new email arrives.
4. Click only the newest link; the old link should be invalid.
5. Verify the account and then try registering the same username again; this time it must be rejected.

These commands create the schema and apply all features. Run them once, in this exact order.

```
cd "C:\textbackslash Users\textbackslash YOUR\_NAME\textbackslash Documents\textbackslash GitHub\textbackslash
$prodDbUrl = Read-Host "Paste the Render External Database URL"
```

Step Run each file

```
psql $prodDbUrl -f db/001_schema.sql
psql $prodDbUrl -f db/002_sample_problems.sql
psql $prodDbUrl -f db/003_admin_features.sql
psql $prodDbUrl -f db/004_submission_options.sql
psql $prodDbUrl -f db/005_account_security.sql
psql $prodDbUrl -f db/006_problem_release_times.sql
psql $prodDbUrl -f db/007_problem_due_times.sql
psql $prodDbUrl -f db/008_default_due_time.sql
psql $prodDbUrl -f db/009_align_seed_season.sql
psql $prodDbUrl -f db/010_hint_responses.sql
```

Step Verify

```
psql $prodDbUrl -c "\\dt"  
psql $prodDbUrl -c "SELECT COUNT(*) AS problems FROM problems; SELECT COUNT(*) AS seasons FROM seasons;"
```

You should see tables including users, problems, submissions, seasons, problem_proposals, and hint_requests.

8 Create the Render backend

1. In Render click **New** → **Web Service**.
2. Select the GitHub repository and click **Connect**.
3. Leave **Root Directory** blank. The project is now at the repository root.
4. Set Runtime to **Node**.
5. Set Build Command to `npm install`.
6. Set Start Command to `npm run start:server`.
7. Click **Create Web Service**.

Step Add backend environment variables

Open the new service, click **Environment** in the sidebar, then click **Add Environment Variable**. Add each row:

NODE_ENV	production
DATABASE_URL	Render PostgreSQL Internal Database URL
JWT_SECRET	A new random value generated locally
FRONTEND_URL	Temporarily <code>https://example.com</code>
TRUST_PROXY	1
SMTP_HOST	smtp.resend.com
SMTP_PORT	587
SMTP_SECURE	false
SMTP_USER	resend
SMTP_PASS	Complete Resend API key
EMAIL_FROM	Purdue Math Club POTW <noreply@purdue-math-potw.com>

Generate the JWT value in PowerShell:

```
[Convert]::ToBase64String([Security.Cryptography.RandomNumberGenerator]::GetBytes(48))
```

Click **Save Changes**; Render redeploys the service.

Step Test the backend

Copy the backend's Render URL, then open:

```
https://YOUR-BACKEND.onrender.com/api/health
```

The page must display `{"status":"ok"}` before you create the frontend.

9 Create the Render frontend

1. In Render click **New** → **Static Site**.
2. Select the same GitHub repository and click **Connect**.
3. Leave **Root Directory** blank.
4. Set Build Command to `npm install && npm run build`.
5. Set Publish Directory to `dist`.
6. Under Environment Variables add `VITE_API_URL`. Set its value to the backend URL:

```
VITE_API_URL=https://YOUR-BACKEND.onrender.com
```

7. Click **Create Static Site**.

Step Add the SPA rewrite

In the frontend service click **Redirects/Rewrites** (or **Settings** → **Redirects and Rewrites**), click **Add Rule**, and enter:

Source	Destination	Action
/*	/index.html	Rewrite

Save the rule and redeploy if Render asks.

10 Connect the two services

Copy the frontend URL, for example `https://potw-frontend.onrender.com`. Return to the backend service, open **Environment**, and replace the temporary value with:

```
FRONTEND_URL=https://YOUR-FRONTEND.onrender.com
```

Save and redeploy the backend. This allows browser requests from the frontend to reach the API.

11 First production test

Run these tests in order:

1. Open the frontend Render URL.
2. Register a test account.
3. Confirm the verification email arrives.
4. Log in and submit a solution.
5. Log in as the admin and open **Admin** → **Grading**.
6. Grade the submission and confirm the leaderboard updates.
7. Open **Hints**, request a hint, and respond from the admin proposals tab.

8. Confirm the solver sees the response.
9. Test file upload, profile history, password reset, and anonymous mode.
10. Refresh `/leaderboard`, `/profile`, and `/hints`; each must still load because of the rewrite.

12 Common problems

No Environment tab	You opened the database or Static Site. Open the backend Web Service .
Emails fail with 403	The domain is not verified, the sender does not use the verified domain, or you used a shortened API key.
CORS error	Set backend <code>FRONTEND_URL</code> to the exact frontend URL, save, and redeploy.
Frontend API errors	Check <code>VITE_API_URL</code> ; it must be the backend URL with no trailing <code>/api</code> .
Refresh gives 404	Add the Static Site rewrite <code>/* → /index.html</code> as a Rewrite.
Database connection fails	Use the Render Internal URL in the backend and confirm the database is Available.
Migrations fail with relation exists	Usually safe; many migrations are idempotent. Read the first actual ERROR line.

13 After launch

- Enable paid PostgreSQL backups/recovery appropriate for your data.
- Rotate the database credential and Resend key if either was exposed.
- Add a custom website domain later; Render's generated URL works without one.
- Keep the backend service running continuously because its scheduler archives expired problems.
- Never put secrets in frontend variables; anything beginning with `VITE_` is visible in the browser build.

14 Useful official documentation

- Render deployment: <https://render.com/docs/your-first-deploy>
- Render Postgres: <https://render.com/docs/postgresql-creating-connecting>
- Resend domains: <https://resend.com/docs/dashboard/domains/introduction>
- Resend API keys: <https://resend.com/docs/dashboard/api-keys/introduction>
- Resend SMTP: <https://resend.com/docs/send-with-smtp>